ICG 2026

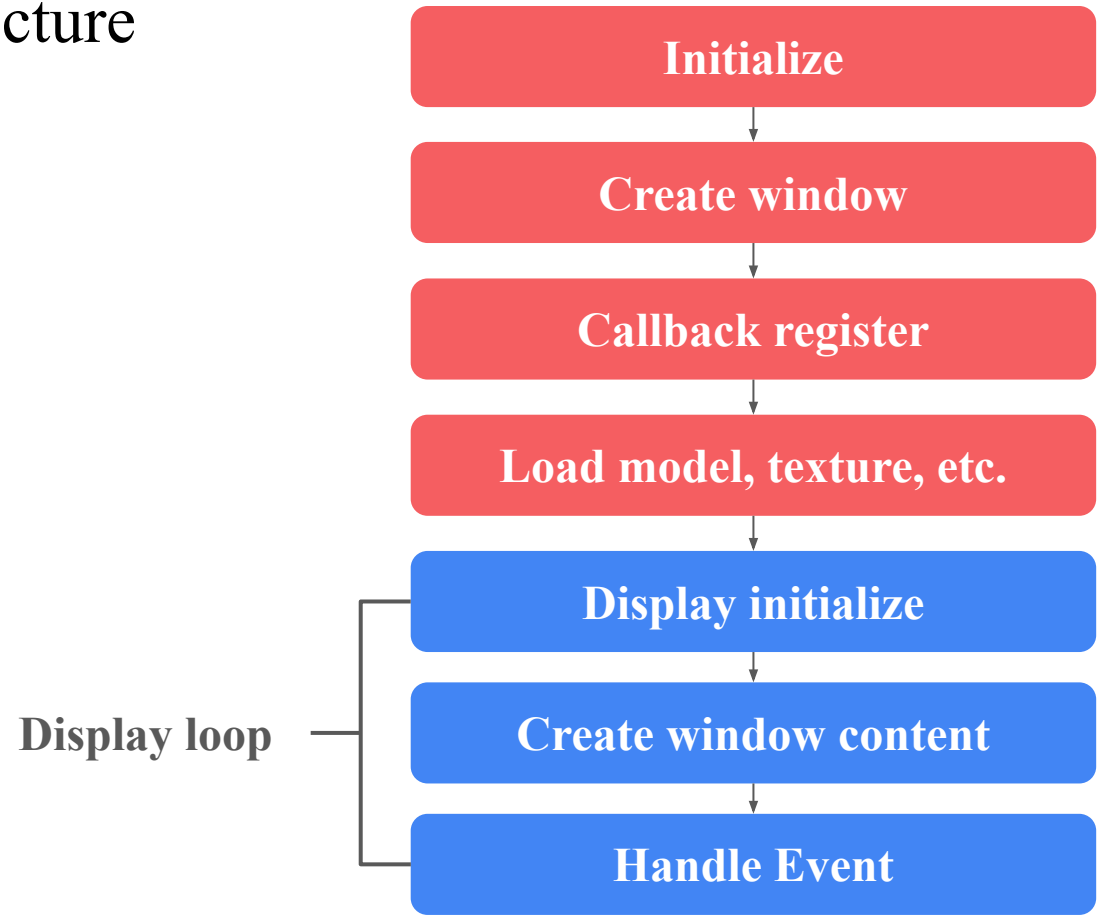
HW1

IDE & Kit

We have already posted the installation guide on E3: “HW0"

- Visual Studio Code
- C++
- CMake
- CMake Tools extension
- GLFW (provided in zip)
  - An open-source, multi-platform library for OpenGL
  - Provides a simple API for creating windows and receiving input.
- GLAD (provided in zip)
  - An OpenGL loading library that loads pointers to OpenGL functions at runtime
- GLM (provided in zip)
  - Math library for OpenGL
- tinyobjloader (provided in zip)
  - obj loader

Architecture



Initialize & Window 1/4

- int glfwInit()
  - Initialize GLFW
  - Return GLFW_TRUE when successful, else GLFW_FALSE
- void glfwWindowHint(int hint, int value)
  - Window settings for the next window creation
  - In this homework, we will use OpenGL 3.3 core profile

```
glfwInit();
glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
```

Initialize & Window$_{2/4}$

❖ GLFWwindow* glfwCreateWindow(int width, int height, const char* title, GLFWmonitor* monitor, GLFWmonitor* share)

- ➢ Create a window with the specified width, height, and title
- ➢ monitor: monitor is used for full-screen mode, NULL for window mode
- ➢ share: the window to share resources with, NULL to not share resources
- ➢ Return the handle of the created window, or NULL if an error occurred

```
GLFWwindow* window = glfwCreateWindow(
    SCR_WIDTH, SCR_HEIGHT, "ICG 2026 HW1 - Outdoor Magic", nullptr, nullptr);

if (!window) {
    std::cerr << "Failed to create GLFW window" << std::endl;
    glfwTerminate();
    return -1;
}
```

Initialize & Window$_{3/4}$

- void glfwMakeContextCurrent(GLFWwindow* window)
  - Make context current for the calling thread
- GLFWframebuffersize glfwSetFramebufferSizeCallback(GLFWwindow* window, GLFWframebuffersizefun cbfun)
  - Register a callback function for window resize
- GLFWkeyfun glfwSetKeyCallback(GLFWwindow* window, GLFWkeyfun cbfun)
  - Register a callback function for key events
- void glfwSwapInterval(int interval)
  - Set the number of screen updates to wait before swapping buffers and returning after calling glfwSwapBuffers()

```
glfwMakeContextCurrent(window);
glfwSetFramebufferSizeCallback(window, framebufferSizeCallback);
glfwSetKeyCallback(window, keyCallback);
glfwSwapInterval(1);
```

Initialize & Window 4/4

- int gladLoadGLLoader((gladloadproc)glfwGetProcAddress))
  - Initialize GLAD to get the OpenGL function pointer

```
if (!gladLoadGLLoader((GLADloadproc)glfwGetProcAddress))
{
    std::cout << "Failed to initialize GLAD" << std::endl;
    return -1;
}
```

Depth test

To prevent occluded faces from being rendered, we need to enable depth testing

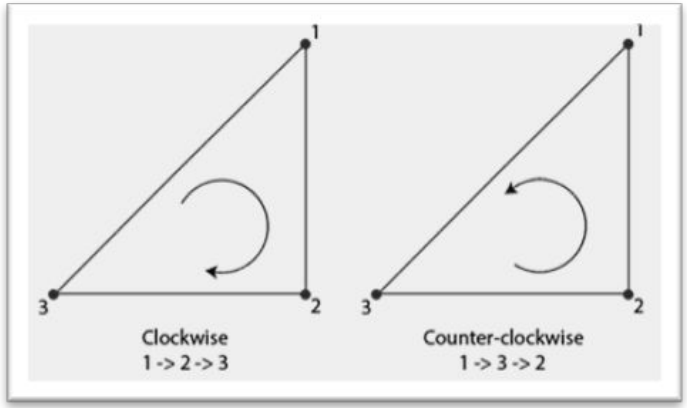
- void glEnable(GL_DEPTH_TEST)
  - While depth test is enabled, OpenGL tests the depth value of each fragment against the content in the depth buffer. If the test passes, the fragment is rendered. If not, the fragment is discarded
- void glDepthFunc(GLenum func)
  - Specify how the test is performed
  - func: GL_NEVER, GL_LESS, GL_EQUAL, GL_LEQUAL, GL_GREATER, GL_GEQUAL, GL_NOTEQUAL, GL_ALWAYS
  - GL_LEQUAL: test passes If the fragment depth ≤ the depth stored in the buffer

```
glEnable(GL_DEPTH_TEST);
glDepthFunc(GL_LEQUAL);
```

Face culling

Face culling reduces the number of faces rendered by discarding faces that are not visible

- void glEnable(GL_CULL_FACE)
  - Tell OpenGL to enable face culling
- void glFrontFace(GLenum mode)
  - mode:GL_CW, GL_CCW
  - Faces with specified ordered vertices are defined as front
- void glCullFace(GLenum mode)
  - mode: GL_FRONT, GL_BACK, GL_FRONT_AND_BACK
  - Cull specified faces




```
glEnable(GL_CULL_FACE);
glFrontFace(GL_CCW);
glCullFace(GL_BACK);
```

Display loop

Before we start to draw, we need to clear the color buffer and the depth buffer

- void glClearColor(GLfloat red, GLfloat green, GLfloat blue, GLfloat alpha)
  - Set the color value that is used to reset the color buffer
- void glClear(GLbitfield mask)
  - Clear the specified buffer
  - mask:
    - GL_COLOR_BUFFER_BIT: clear color buffer
    - GL_DEPTH_BUFFER_BIT: clear depth buffer

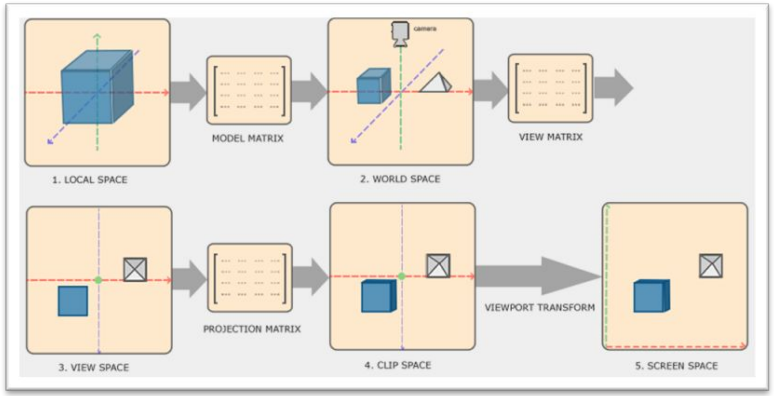
```
glClearColor(153/255.0, 204/255.0, 255/255.0, 1.0f);
glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
```

Draw a model$_{1/2}$

- void drawModel(const string& **type**, const glm::mat4& **model**,
  const glm::mat4& **view**, const glm::mat4& **projection**,
  const glm::vec3& **color,** float **alpha =** 1.0f)
  - **Draw the target model**
  - type: the target model (“cube”, "magic_ball", “bird”,”flower”)
  - model / view/ projection: model /view/ projection matrix
  - color r/ g/ b: red/ green/ blue color (float 0~1)
  - alpha : opacity (float 0 ~ 1)
    - Optional. If omitted, alpha = 1.0f (fully opaque).

```
drawModel("cube",
          model, view, projection,
          glm::vec3(0.9f, 0.8f, 0.6f));
```

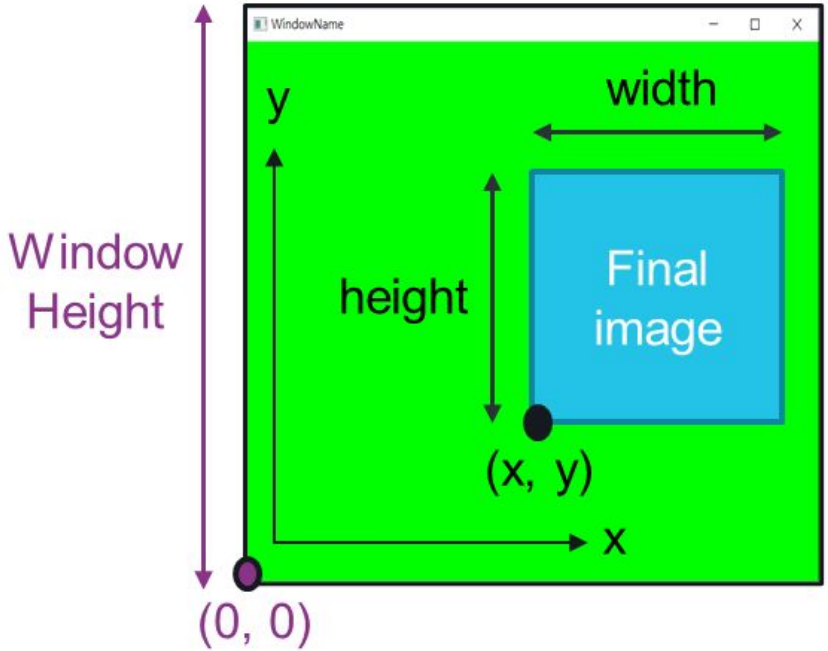
```
drawModel("cube",
          mainModel, view, projection,
          glm::vec3(1.0f, 0.8f, 0.12f), mainAlpha);
```



Draw a model$_{2/2}$

- void glViewport(GLint x,GLint y, GLint width, GLint height)
  - Specify the viewport rectangle

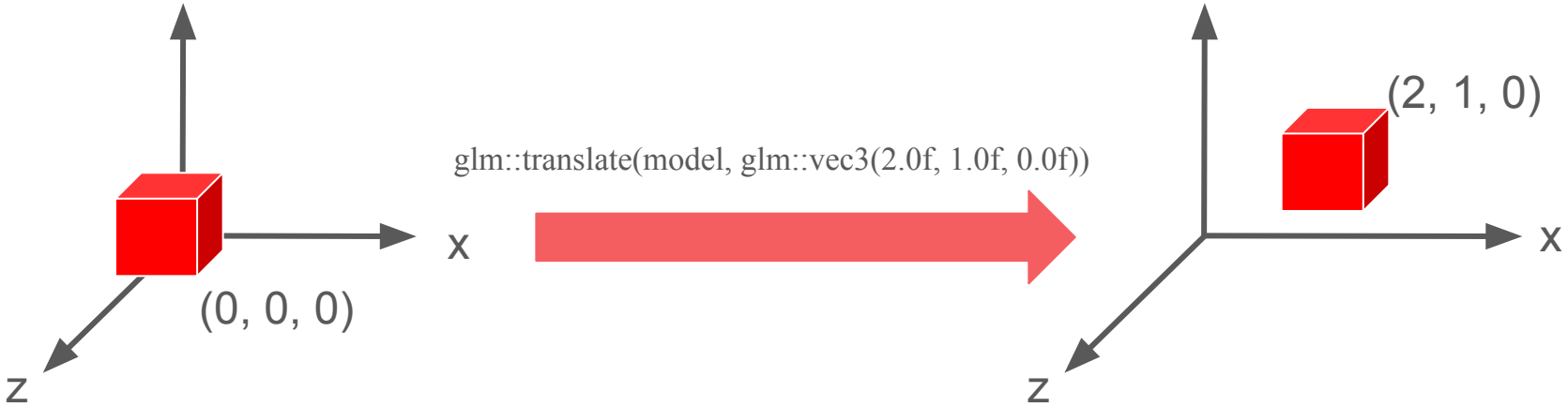
```
glViewport(0, 0, SCR_WIDTH, SCR_HEIGHT);
```



Model matrix 1/3

- glm::translate(glm::mat4 M, glm::vec3 translation)
  - Return M * (translation matrix)

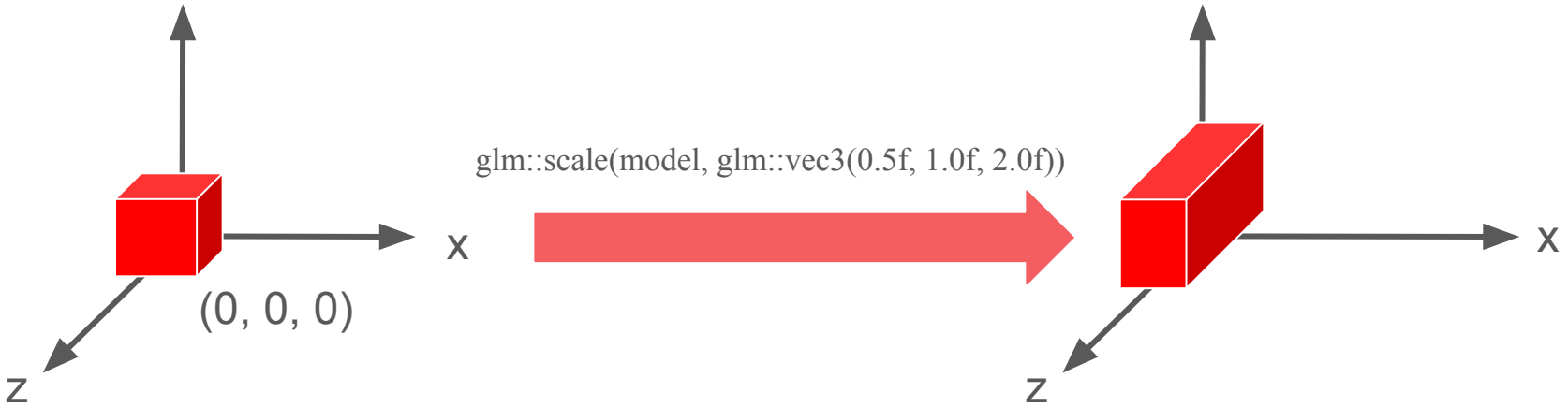
```
glm::mat4 model(1.0f);
model = glm::translate(model,glm::vec3(2.0f, 1.0f, 0.0f));
drawModel("cube",model,view,projection,glm::vec3(1.0f, 0.0f, 0.0f));
```



Model matrix$_{2/3}$

- glm::scale(glm::mat4 M, glm::vec3 scale)
  - Return M * (scale matrix)

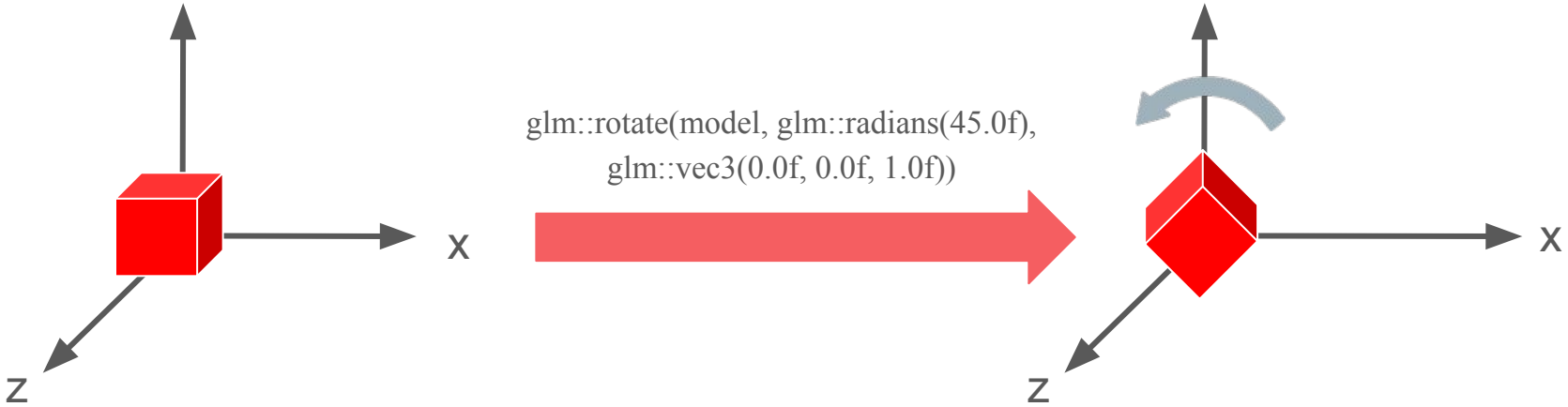
```
glm::mat4 model(1.0f);
model = glm::scale(model,glm::vec3(0.5f, 1.0f, 2.0f));
drawModel("cube",model,view,projection,glm::vec3(1.0f, 0.0f, 0.0f));
```



Model matrix$_{3/3}$

```
glm::mat4 model(1.0f);
model = glm::rotate(model,glm::radians(45.0f),glm::vec3(0.0f, 0.0f, 1.0f));
drawModel("cube",model,view,projection,glm::vec3(1.0f, 0.0f, 0.0f));
```

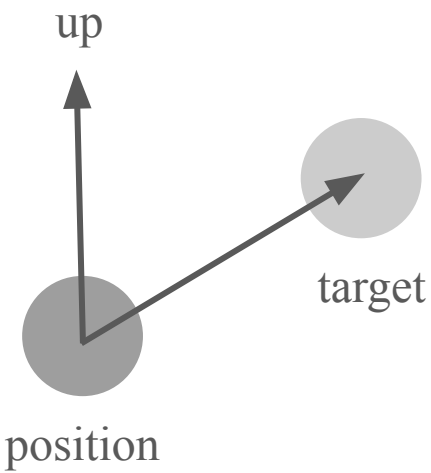
- glm::rotate(glm::mat4 M, GLfloat angle, glm::vec3 axis)
  - Return M * (rotation matrix)
  - The rotation matrix rotates an angle in radians about the given axis
- glm::radians(GLfloat degree)
  - Convert the given degree to radian



View matrix

- glm::lookAt(glm::vec3 position, glm::vec3 target, glm::vec3 up)
  - Return view matrix with camera at position looking at the target with up vector

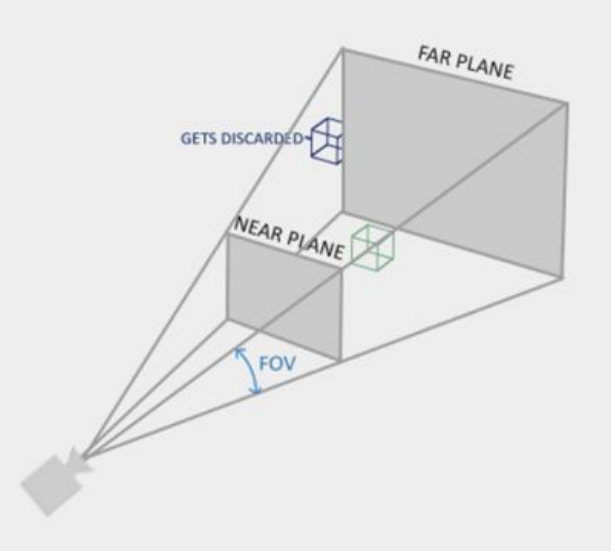
```
glm::mat4 view = glm::lookAt(
                    glm::vec3(0.0f, 50.0f, 90.0f),
                    glm::vec3(0.0f, 0.0f, 0.0f),
                    glm::vec3(0.0f, 1.0f, 0.0f));
```



Projection matrix

- glm::perspective(GLfloat fov, GLfloat aspect, GLfloat near, GLfloat far)
  - Return the perspective projection matrix with the above parameters
  - fov: specify Field of View in radians
  - aspect: specify aspect ratio of the scene
  - near: specify near plane
  - far: specify far plane
  - Coordinates in front of near plane or behind far plane will not be drawn

```
glm::mat4 projection = glm::perspective(
                       glm::radians(45.0f),
                       (float)SCR_WIDTH / (float)SCR_HEIGHT,
                       0.1f,
                       1000.0f);
```



Display loop

- void glfwSwapBuffers(GLFWwindow* window)
  - Swap buffer at the end of the display loop
- void glfwPollEvents()
  - Handle any events occurring while rendering the frame

```
glfwSwapBuffers(window);
glfwPollEvents();
```

Key callback

```
void keyCallback(GLFWwindow* window, int key, int scancode, int action, int mods)
{
    if (key == GLFW_KEY_ESCAPE && action == GLFW_PRESS)
        glfwSetWindowShouldClose(window, true);
}
```

- The above function is registered for a key callback. We can check for key events and act correspondingly
- It sets glfwWindowShouldClose to true when the escape key is pressed, which will exit the display loop.
- The full list of keys can be found here.
- The full list of actions and their effects can be found here.

Homework 1 - Introduction

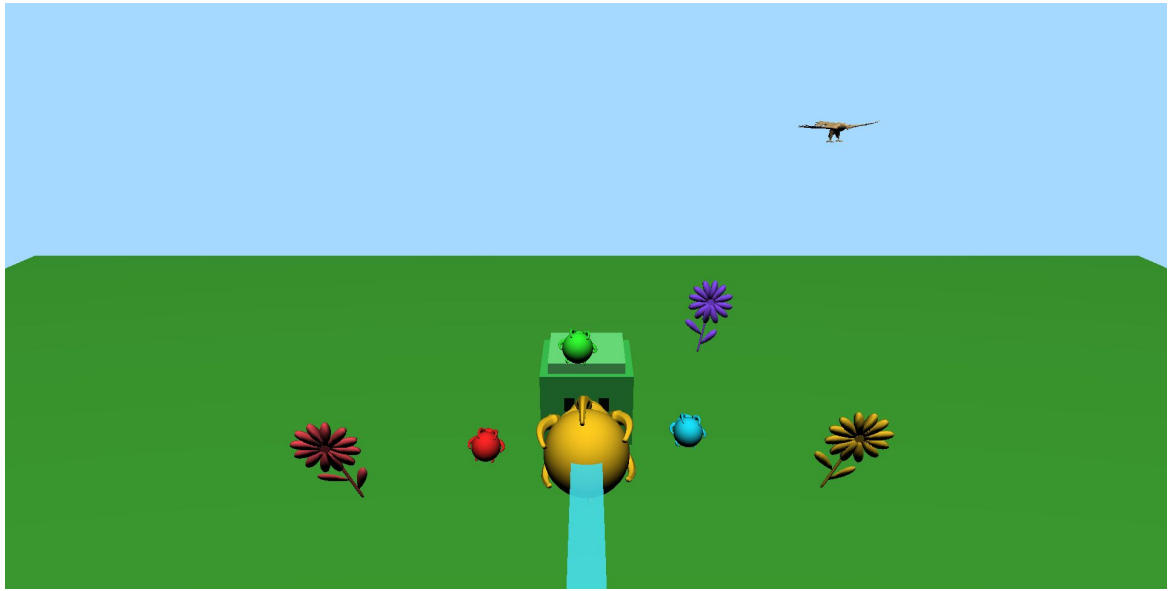
Story

I have been reincarnated. In my previous life, I was a legendary ICG Master, feared and respected for mastering all kinds of magic. But now, I have been reborn as a slime??!! It does not matter. I swear that in this life, I will reclaim everything that once belonged to me. To become a powerful wizard again, I must recover my lost abilities, restore my magic balls, and relearn how to cast spells.



Demo

- https://youtu.be/qEfdUkBFkvs



Requirement 1/6 — Scene Setup

Camera:

Position: (0, 14, 23)

Target: (0, 5, 0)

Up: (0, 1, 0)

Fov: 45

near: 0.1

far: 1000

Scene Objects :

`cube.obj`

`bird.obj`

`magic_ball.obj`

Three `flower.obj`

Ground (cube) :

Position: (0, -0.5, 0)

Scale: (70, 1, 40)

Color: (0.25, 0.65, 0.20)

Main character

Initial Root Position: (0, 0, 0)

At least 3 cubes

Hierarchical transformation required

You can be creative in your character design!

Bird

Base Position: (0, 10, -5)

Scale: (0.2, 0.2, 0.2)

The base position is the center of the elliptical flight path.

flower 1

Position: (-7, 0, 4.5)

Scale: (1.35, 1.35, 1.35)

Color: adjustable

flower 2

Position: (7.5, 0, 4)

Scale: (1.35, 1.35, 1.35)

Color: adjustable

flower 3

Position: (5, 0, -6)

Scale: (1.35, 1.35, 1.35)

Color: adjustable

Requirement $_{2/6}$— Main Character

Main character (Must contain at least 3 cubes) :

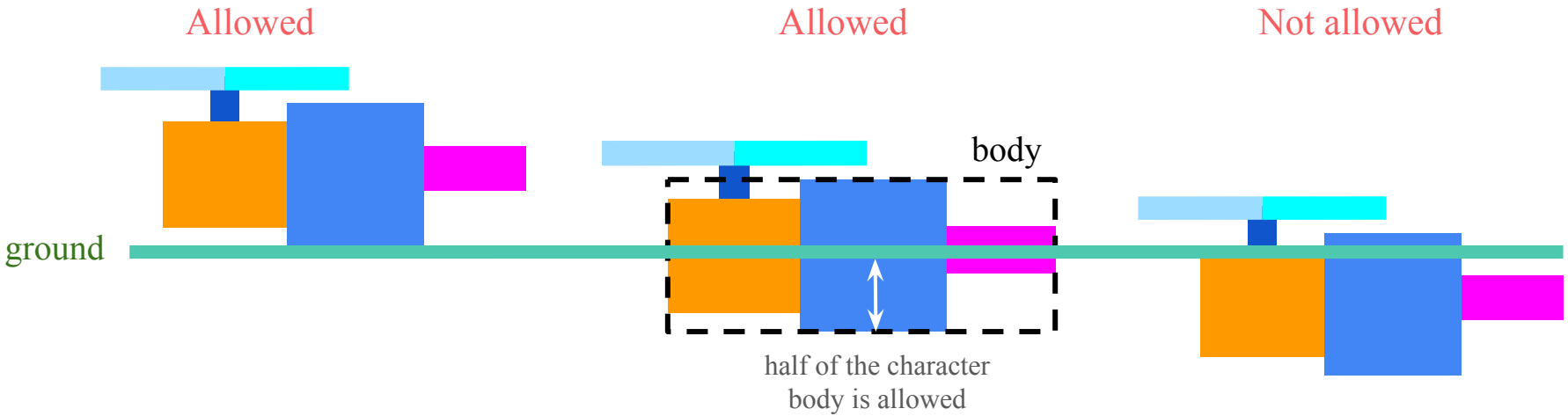
- Must contain at least 3 cubes.
- Cubes must use hierarchical transformation.
- The entire slime should move and rotate through a shared root transformation.
- The facing direction must be clearly distinguishable.
- You can be creative in your design!



Requirement $_{3/6}$— Movement & Boundary

Boundary collision detection:

- The character cannot move below the ground.
- An error margin of less than half of the body is allowed.



Requirement 3/6 — Movement & Boundary

Keyboard input:

- ❖ W : Move forward
- ❖ S : Move backward
- ❖ A : Move left
- ❖ D : Move right
- ❖ Space : Move upward
- ❖ Shift : Move downward
  - ➢ W / S / A / D use world-space directions.
  - ➢ The slime faces its movement direction.

- **Please use deltaTime for frame-rate-independent movement.**

$$\text{position} += \Delta t \cdot \text{speed}$$

$$\Delta t = CurrFrameTimestamp - lastFrameTimestamp$$

- ❖ Slime Hopping
  - ➢ When moving horizontally on the ground:
  - ➢ Hopping is not required while the slime is flying.

$$y_{offset} = H|\sin(\omega t)|$$

`H` : hop height
`ω` : hop speed
`t` : elapsed movement time

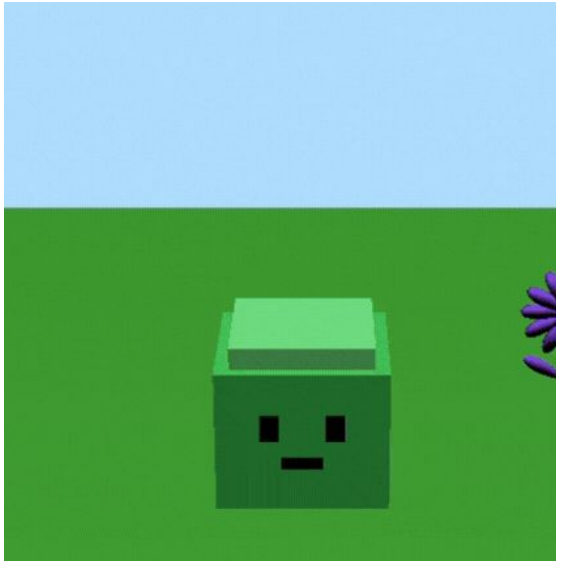
Requirement $_{4/6}$— Magic Casting

Magic Ball :

- Magic Casting
  - **Press R to cast the spell.**
- Main Magic Ball
  - Appears above the slime
  - Gradually appears: alpha 0 → 1
  - Rotates around its own center
- Four Small Magic Balls
  - Appear around the main magic ball.
  - Gradually appear: alpha 0 → 1
  - The four small balls should use visibly different colors.
  - Each ball rotates around its own center.
  - The four small balls orbit around the main magic ball.



Requirement 4/6 — Magic Casting



Magic ball :

- Magic ball Hierarchy
  - The four small balls are children of the main magic ball
  - Each small ball inherits the transformation of the main ball
  - Each small ball has two different rotations:
    - Self rotation — rotates around its own center
    - Orbit rotation — rotates around the main ball
  - The four small balls remain at a fixed distance from the main ball
  - Do not calculate the small-ball positions independently in world coordinates
  - The entire hierarchy should share the same parent transformation.

$$M_{small} = M_{main} R_{orbit} T_{offset} R_{self} S$$

`Rorbit` : orbit around main ball
`T` : offset from main ball
`Rself` : rotate around itself

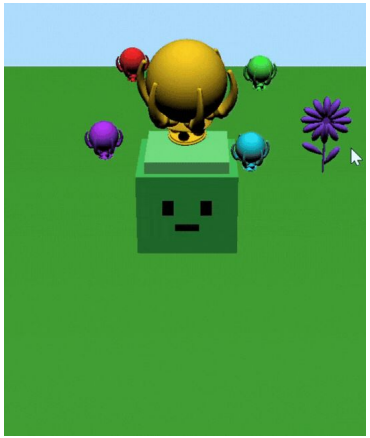
Requirement $_{4/6}$— Magic Casting

Move to Front :

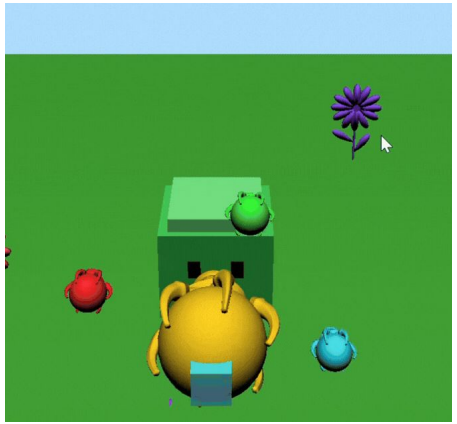
- Before firing, move the entire magic-ball hierarchy from above the slime to the front of the slime.
- The main ball and all four small balls must move together.
- During Move-to-Front, rotate the entire hierarchy from a horizontal orientation to a vertical orientation.
- Therefore, the small-ball orbit plane changes from horizontal to vertical.

$$\theta = 90^\circ \cdot t$$

$$P_{magic} = (1-t)P_{above} + tP_{front}$$



Requirement 4/6 — Magic Casting



Magic Beam :

- The beam is fired after the magic-ball hierarchy reaches the front.
- The beam starts from the center of the main magic ball.
- The beam should extend along the magic ball's forward direction.
- The magic ball's forward direction should align with the slime's facing direction.
- Beam length should gradually increase.
- Use a cube to create the beam.
- Use translation + scale.

$$BeamDirection = MagicForward$$

$$BeamCenter = MagicPosition + BeamDirection \times \frac{BeamLength}{2}$$

Requirement 4/6 — Magic Casting

Spell Requirements :

Press R

↓

Main + Small Magic balls fade in

↓

Main ball: Self Rotation

Small balls: Self Rotation + Orbit

↓

Move / Rotate the Whole Hierarchy to the Front

↓

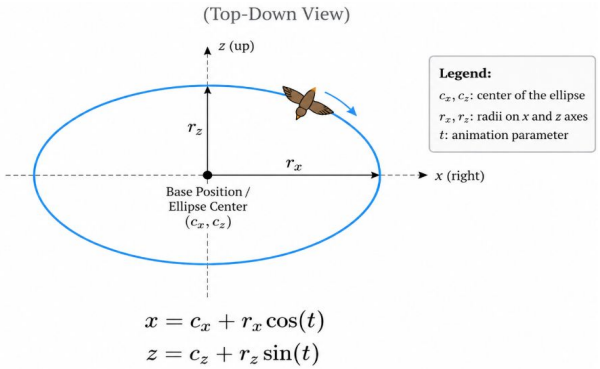
Magic Beam

- Animation speed and timing are up to you.
- Pressing R while a spell is active should not restart the animation.
- You are encouraged to add additional effects in Advanced.

Each grading item is evaluated independently.

Requirement 5/6 — Bird Elliptical Flight

Bird Animation :



- ❖ Elliptical Path
  - ➢ The bird must continuously fly along an elliptical path.
  - ➢ The center of the ellipse is the bird's base position.
  - ➢ The bird position can be calculated as:

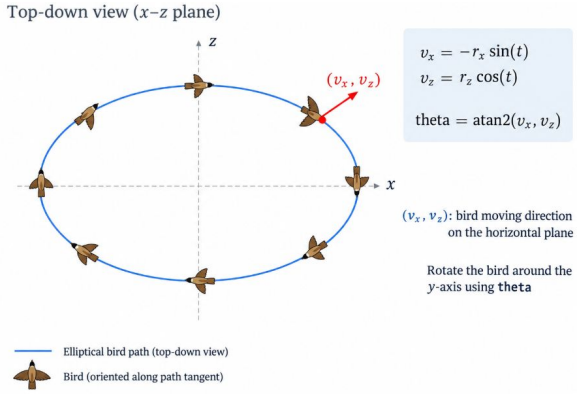
$$x = c_x + r_x \cos(t)$$

$$z = c_z + r_z \sin(t)$$

`c_x,c_z:` center of the ellipse
`r_x,r_z:` radii along the x-axis and z-axis
`t:` animation parameter updated over time

  - ➢ The bird's height may remain constant.
  - ➢ The exact ellipse size and flying speed are up to you.
  - ➢ The bird's base position (0, 10, -5) is used as the center of the ellipse.

Requirement 5/6— Bird Elliptical Flight



Bird Animation :

- Facing Direction
  - The bird should face approximately toward its moving direction.
  - The moving direction can be estimated from the tangent of the elliptical path.
  - For the elliptical path above, the tangent direction can be written as:

$$v_x = -r_x \sin(t)$$

$$v_z = r_z \cos(t)$$

`(v_x, v_z)` represents the bird's moving direction on the horizontal plane.

  - The facing angle can be calculated using:

$$\theta = \text{atan2}(v_x, v_z)$$

Rotate the bird around the y-axis by θ.

.

Requirement 6/6 — Flower Reaction

Flower Reaction :

- Render exactly three flowers using flower.obj.
- The three flowers must use visibly different colors.
- When the spell is cast, the flowers should gradually fall / tilt.
- During recovery, the flowers should gradually return to their original upright pose.
- Rotate each flower around its bottom / base.

Before



Firing



Recovering



Requirement $_{6/6}$— Flower Reaction

Flower Reaction :

- The falling direction should depend on the relative position between the slime and each flower.
- First, calculate the horizontal direction from the slime to the flower:

```
glm::vec3 awayDirection =
    FLOWER_POSITIONS[i] -character.position;
```

- Ignore the vertical component when determining the falling direction:
- Each flower should generally fall away from the slime.
- During recovery, each flower should rotate back toward its original upright orientation.
- Rotate around the bottom / base of the flower, not its center.

$$M_{flower} = T_{position} R_{tilt} S$$

Score - Basic - 90%

Each grading item is evaluated independently.

Depth testing (pass if or equal) – 5%

Face culling (counter-clockwise as front, cull back) – 5%

Camera and perspective – 5%

Scene setup – 10%

Slime character & hierarchy – 10%

Movement, hopping & boundary – 10%

Bird elliptical flight – 10%

Main + small magic balls & hierarchy – 10%

Move-to-Front & Magic beam – 15%

Flower reaction – 10%

Score - Advanced - 10%

Creative Slime Design / Animation – 5%

Creative Spell Effects – 5%

e.g. More expressive hopping / squash & stretch , Energy gathering effects , Particles / sparks

Show us your creativity！ ！ ！

Advanced scores are given on a relative scale, so only the very best can earn a perfect score.

Homework 1 - Submission

- Deadline: 2026/10/11 23:59:59
  - Final score = original score × 0.8 for submissions up to one week late (10/12–10/18).
  - Final score = original score × 0.6 for submissions one to two weeks late (10/19–10/25).
  - Final score = original score × 0.5 for submissions more than two weeks late (after 10/25). No further penalty will be applied.
- Zip and upload the main.cpp and demo.mp4/video.txt on E3
  - The demo video should demonstrate the above criteria while providing a brief description of the displayed score.
- Zip name : studentID_HW1.zip
  - e.g.

    315551000_HW1.zip

    |- main.cpp

    |- demo.mp4/video.txt

Reference

- https://learnopengl.com/
- https://www.glfw.org/documentation